

Case Técnico — Router de Queries & Seleção de Tools

Gabriel Torres

Como usar: a Parte 1 é a história, se pedirem “me conta o que você fez”. A Parte 2 é para quando aprofundarem nas métricas. A Parte 3 são respostas prontas para as perguntas mais prováveis. A Parte 4 é a pergunta que quase sempre vem no fim.

Os números que eu preciso saber de cor

	Valor	Contra o quê
Acurácia do router	100%	chutar a classe majoritária dá 66,7%
o mesmo, com as personas	84%	150 mensagens fora do padrão
Precision@2 do retriever	85%	similaridade pura dá 25%
destes, com a tool certa em 1º	70%	os outros 15% vêm em 2º lugar
o mesmo, com as personas	57,5%	é o número que eu levo para uma reunião
Economia de custo	77,8%	baseline = tudo para o LLM caro
Economia de latência	97,7%	65,3% comparando escopo igual
Testes automatizados	55	2 vieram no case, 53 eu escrevi

Contexto dos dados: 285 ferramentas no catálogo · 53 exemplos de treino · 30 mensagens de avaliação.

A abertura, em 30 segundos

“Implementei os três pilares com ML clássico rodando local — o case pede para decidir o caminho mais barato antes de chamar um LLM caro, então colocar um LLM nessa decisão seria pagar o custo que ela existe para evitar. O router acerta 100% e a busca de ferramentas 85%. Mas o número que eu acho mais interessante é outro: quando testei com mensagens escritas fora do padrão, os 85% viraram 57,5%. Foi isso que me fez construir e medir duas arquiteturas alternativas.”

Parte 1 — A história, em cinco movimentos

1. Comecei pela ideia errada, e o número me corrigiu

Minha primeira ideia foi um orquestrador: um LLM recebendo a mensagem e decidindo se ela é simples ou complexa. Parecia moderno.

Mas o enunciado pede exatamente para escolher o caminho mais barato ANTES de chamar um LLM caro. Colocar um LLM ali seria pagar o custo que a decisão existe para evitar.

Então testei a alternativa barata antes de descartá-la: TF-IDF de caractere com regressão logística, treinado com os 53 exemplos. **Acertou 100% do conjunto de avaliação, em cerca de 1 ms e custo zero.** Não havia ganho a capturar — a decisão se resolveu sozinha.

Detalhe que costuma render pergunta: usei n-gramas de caractere e não de palavra. Português tem muita flexão (fatura/faturas, parcelar/parcelamento) e 53 exemplos é pouco para o modelo ver cada variação. Caractere pega o radical sem stemmer — e de quebra tolera erro de digitação, o que importou muito depois.

2. O problema real estava no lugar em que eu não olhei

Achei que o Pilar 2 seria trivial: vetorizar as 285 ferramentas, medir similaridade, devolver as duas melhores. Rodei e deu 25%.

Fui investigar e o catálogo tem uma armadilha proposital — quase-duplicatas. Para várias mensagens, a ferramenta mais parecida não é a correta:

Mensagem	O que a busca devolve	O que era esperado
“Manda o pdf da minha fatura atual”	enviar_pdf_fatura_atual	consultar_fatura
“Quanto eu tenho disponível na conta agora?”	consultar_valor_disponivel_conta	consultar_saldo

A variante longa vence só porque repete as palavras do cliente. **Não é motor de busca ruim — é que “mais parecido” e “correto” são coisas diferentes aqui.**

3. A correção: preferir o genérico

Duas etapas. Primeiro fundo duas buscas — uma por caractere, outra por palavra — usando Reciprocal Rank Fusion, que combina as POSIÇÕES e não as notas, então não preciso normalizar escalas diferentes. Depois aplico uma penalidade proporcional a quão específico é o nome da ferramenta.

O raciocínio: entre duas ferramentas que atendem à mesma intenção, a mais geral costuma ser a porta de entrada certa. As de nome longo são refinamentos de um catálogo mal higienizado.

Estratégia	Precision@2
TF-IDF de palavra, similaridade pura	0,25
TF-IDF de caractere, similaridade pura	0,35
Fusão dos dois (RRF)	0,45
+ prior de generalidade	0,85

Testei e descartei duas alternativas, e vale ter os números: **BM25** com normalização agressiva de comprimento deu 0,20 — o parâmetro b normaliza saturação de frequência, não especificidade de conceito, então ele nem ataca o problema. **Clusterizar** as quase-duplicatas deu 0,60 — a transitividade encadeia grupos que não deveriam se juntar e derruba o recall.

4. O teste de fogo

O conjunto de avaliação tem 30 mensagens bem comportadas. Cliente de banco não escreve assim. Então montei uma bateria de subagentes com o SDK da OpenAI para atacar a própria solução.

Cinco personas geraram 150 mensagens novas. O resultado foi o achado mais útil do projeto:

Persona	Como escreve	Precision@2
leigo_idoso	frases longas, educadas, sem termo técnico	40%
caotico	pedidos misturados, muda de assunto	50%
jovem_gurias	abreviação, gíria, sem pontuação	56%
especialista_bancario	jargão formal e correto	68%
erros_digitacao	erro de digitação em quase toda palavra	70,8%
Média		57,5%

O ponto que eu faria questão de dizer: a degradação não é uniforme, ela tem endereço. E o pior caso é o idoso leigo — justamente o perfil que mais depende do atendimento funcionar.

Usei também dois subagentes como avaliadores, com o papel de criticar a solução: um olhando produção (segurança, resiliência, latência) e outro olhando método (viés, vazamento, validade das métricas). O de método achou o ponto mais desconfortável: 40% do conjunto oficial de avaliação tem sobreposição alta com o de treino, e 3 mensagens são cópias literais.

5. A decisão

Com o problema medido, construí duas alternativas e rodei as mesmas 180 mensagens nas três versões:

Versão	O que faz	P@2 nas personas	Latência
V1 — local	classificador + busca, sem LLM	57,5%	2 ms
V1.0.1 — LLM	1 chamada decide rota e ferramenta	92%	1.107 ms
V1.0.2 — híbrida	local resolve o certo, LLM o resto	92%	971 ms

Entreguei a V1 no case porque ela cumpre o que foi pedido sem depender de rede nem de chave de API — e porque um router por LLM quebraria o próprio harness do Pilar 3, que precifica a chamada em US\$ 0,010 fixos e simula a latência com sleep. As outras duas estão documentadas ao lado, com os números da comparação.

Parte 2 — As métricas, explicadas

O enunciado pede três métricas. Eu entreguei essas três e acrescentei seis, porque cada uma das obrigatórias, sozinha, conta meia verdade.

As três obrigatórias

Métrica	Valor	O que ela realmente diz
Acurácia do router + matriz de confusão	100% (30/30)	Acertou todas as 30. A matriz mostra 10/10 FAST_PATH e 20/20 AGENT, zero erro nas duas direções.
Precision@2 do retriever	85% (17/20)	A ferramenta certa estava entre as duas devolvidas em 17 das 20 mensagens que exigiam ferramenta.
Economia de custo e latência	77,8% e 97,7%	Contra o baseline definido pelo próprio case: mandar toda mensagem ao LLM caro com as 285 ferramentas no prompt.

As seis que eu acrescentei

Métrica	Valor	Por que ela existe
IC 95% da acurácia (Wilson)	[88,6%, 100%]	Com n=30, dizer “100%” sem intervalo é enganoso.
Erro assimétrico do router	0 e 0	Mandar AGENT ao FAST_PATH deixa o cliente sem atendimento. O contrário só gasta dinheiro. São erros de gravidade diferente e não podem ser somados.
MRR@2	0,775	Mostra se acertei em 1º lugar ou por sorte no 2º.
Sucesso ponta a ponta	85%	Precision@K ignora quem o router já errou. Esta exige acertar rota E ferramenta.
Latência p50/p95 por estágio	router 0,9 ms · busca 1,7 ms	Média esconde cauda, e SLA se mede na cauda.
Economia de latência comparável	65,3%	A fórmula obrigatória compara escopos diferentes. Esta compara igual.

As três ressalvas — diga antes que perguntem

Isto é o que eu levaria como postura: apontar o que enfraquece meus próprios números antes que alguém aponte por mim.

- **Os 97,7% de latência comparam coisas diferentes.** Do meu lado o harness soma só router + busca; do lado do baseline, soma a chamada de LLM. Com a chamada nos dois lados, a economia real é 65,3%. Reporto os dois: o primeiro porque é a fórmula que o enunciado pede, o segundo porque é o em que eu acredito.
- **Precision@2 só conta as 20 mensagens que o router mandou para o agente.** Um router ruim poderia inflar essa métrica escondendo os próprios erros. Foi por isso que criei o sucesso ponta a ponta.
- **40% do conjunto de avaliação se sobrepõe ao treino, com 3 cópias literais.** Nas 18 mensagens restantes a acurácia se mantém em 100% — mas parte do meu 100% original era o modelo reconhecendo algo que já tinha visto.

O detalhe que quase ninguém pergunta, mas que impressiona

Precision@2 é “a certa estava entre as duas”. Se eu abrir esse número por posição, ele fica mais informativo:

O que aconteceu	Quantas	%
Ferramenta certa em 1º lugar	14	70%
Ferramenta certa em 2º lugar	3	15%
Ferramenta certa não apareceu	3	15%

Isso corta nos dois sentidos, e eu diria os dois. Se o agente executasse cegamente a primeira sugestão, o número seria 70%, não 85% — os 85% dependem do desenho passar duas opções, que é o que o enunciado pede com $k=2$. Por outro lado, 70% em primeiro lugar mostra convicção, não sorte.

E os 3 que erra têm todos a mesma causa: a ferramenta certa nem entrou nas 15 candidatas do primeiro estágio. É falha de recall, que é o teto de tudo que vem depois — nenhuma reordenação salvaria. Está documentado e tem teste dedicado, para não virar surpresa.

Parte 3 — Perguntas prováveis

“Por que você não usou LLM? Hoje todo mundo usa.”

O enunciado não proíbe — ele diz duas vezes que a técnica é livre. Mas o requirements.txt tem quatro pacotes e nenhum cliente HTTP, e o common/mock_llm.py simula custo e latência da chamada. O case é fechado por construção.

Além disso, um router por LLM invalidaria o próprio harness do Pilar 3: ele precifica a chamada em US\$ 0,010 fixos e simula latência com sleep. Uma chamada real traria números que essa contabilidade não modela.

Mas eu testei a alternativa mesmo assim, fora do entregável, para saber quanto estava deixando na mesa. A resposta está medida: **57,5% contra 92% nas personas.**

“100% de acurácia em 30 mensagens não é overfitting?”

Provavelmente há um pouco, e eu mesmo achei isso — por dois caminhos. Primeiro, reporto o intervalo de confiança de Wilson: [88,6%, 100%]. Com n=30, 100% seco não é conclusivo.

Segundo, um dos avaliadores encontrou que 40% do conjunto de avaliação se sobrepõe ao treino, com 3 cópias literais. Refiz a conta nas 18 mensagens sem sobreposição e a acurácia se manteve em 100%. E o teste com personas, que é fora da distribuição, derrubou para 84% — o que é o comportamento honesto de um modelo pequeno.

“De onde vieram os baselines que você compara?”

Têm origens diferentes, e é bom eu ser exato. Os 25% de similaridade pura eu medi, rodando TF-IDF de palavra sozinho. Os 0% de economia vêm do próprio case, que define o baseline no mock_llm.py em US\$ 0,030 por mensagem.

Já os 66,7% do router eu calculei, não medi: 20 das 30 mensagens são AGENT, então um return fixo acerta 20/30. É o baseline de classe majoritária, que se calcula a partir da distribuição.

“85% é bom?”

Depende do que está do outro lado. Contra similaridade pura, que é a solução óbvia, é 25% — então são 60 pontos de ganho. Contra escolher ao acaso entre 285 ferramentas, é 0,7%.

Mas fora do conjunto do case são 57,5%, e é esse o número que eu levaria para uma decisão de produto. O conjunto oficial é mais fácil que a realidade.

“Você validaria isso em produção?”

Não como está. Falta a camada que eu considero mais importante: nenhuma política entre escolher a ferramenta e executá-la. Num dos casos de teste o sistema devolveu pix_enviar em primeiro lugar para uma pergunta sobre segunda via de cartão. No case não tem consequência porque o mock só imprime texto — em produção, uma ferramenta que movimenta dinheiro não pode ser selecionada por similaridade textual sem confirmação.

E falta um conjunto de validação com mensagens de cliente real, rotuladas por pessoa.

“Como você garantiu que suas próprias métricas estão certas?”

Escrevi 53 testes para o código de medição, além dos 2 que já vinham no case. Incluindo casos que costumam quebrar em silêncio: intervalo de Wilson quando a proporção é 0 ou 1, matriz de confusão degenerada, divisão por zero na economia, e um teste que documenta as 3 falhas conhecidas do retriever.

O raciocínio é simples: se o harness estiver errado, todas as outras métricas viram ficção.

Parte 4 — O que eu faria diferente

Esta é a pergunta que quase sempre vem no fim, e é onde dá para mostrar mais maturidade do que na parte técnica.

1. Construiria o conjunto difícil ANTES de otimizar

Foi meu maior erro de ordem. Eu ajustei o parâmetro do prior olhando o conjunto oficial e só depois descobri, com as personas, que aquele conjunto é fácil. Ou seja: otimizei contra a régua errada e só fui saber no fim.

O certo seria gerar as personas no primeiro dia e usá-las como conjunto de desenvolvimento, deixando o oficial intocado até a medição final.

2. Checaria vazamento entre treino e avaliação no primeiro dia

Os 40% de sobreposição foram achados por um avaliador, no fim. É uma verificação de dez minutos que muda a interpretação de todos os números de router. Deveria ser o primeiro comando que eu rodo em qualquer projeto com dados prontos.

3. Mediria o teto antes de trabalhar no que vem depois

Passei tempo refinando a reordenação quando 15% das falhas são de recall — a ferramenta certa nem chega a ser candidata, e nenhuma reordenação resolve isso. Medir Recall@15 no começo teria me dito onde o esforço valia a pena.

4. Rodaria as comparações mais de uma vez

Quando comparei a versão com LLM, rodei uma vez. Depois descobri que o mesmo código, com a mesma entrada e temperatura zero, deu 75%, 85% e 95% em três execuções. Uma rodada única não sustenta diferenças menores que uns 10 pontos — e eu quase concluí algo a partir de ruído.

5. Colocaria a camada de política desde o começo

Deixei a segurança como “item documentado”. Se fosse produção, a separação entre selecionar e executar ferramenta sensível seria requisito de arquitetura, não melhoria futura. É mais barato desenhar assim do que enxertar depois.

O que ficou aberto, e eu diria sem ser perguntado

- Nenhuma política entre escolher e executar ferramenta financeira.
- Nenhum conjunto meu é grande, humano e limpo ao mesmo tempo: o oficial é humano mas pequeno e contaminado; o das personas é grande e limpo mas sintético.
- Validação de tipo: passar None ou um número para o router levanta exceção não tratada.
- Precision@2 não é sucesso do cliente — eu não meço qual ferramenta foi de fato executada, nem com quais argumentos.

A frase de fechamento

“Se eu tivesse que resumir: o case me deu 85%, e o trabalho que eu considero de verdade foi descobrir que esses 85% valiam 57,5% quando o usuário escreve como gente. O resto — as duas arquiteturas alternativas, os testes de medição, as ressalvas que eu mesmo levantei — saiu dessa descoberta.”